

UNIDAD 8

Diseño de **Software** Avanzado

Principios, Patrones y Experiencia de Usuario

Agenda de la Sesión – Parte I

8.1 Introducción al Diseño

8.1.1 Conceptos Fundamentales

8.1.2 Diseño vs Requisitos

8.1.3 Acoplamiento y Cohesión

8.1.3 Principios SOLID

8.2 Estrategias de Diseño

8.3 Diseño Orientado a Objetos

8.4 Diseño Orientado a Datos

8.5 Trazabilidad en Arquitectura

8.6 Introducción a Patrones

Agenda de la Sesión – Parte II

8.6.1 Patrones Creacionales

8.6.2 Patrones Estructurales

8.6.3 Patrones de Comportamiento

8.7 Sistemas Interactivos

8.7.1 Diagramas de Secuencia

8.7.2 Máquinas de Estado

8.8 Sistemas por Eventos

8.9 Experiencia de Usuario (UX)

8.9.1 Técnicas Diseño Visual

8.9.1.1 Diseño Inclusivo

Agenda de la Sesión – Parte III

8.9.1.2 Interfaces de Voz

8.9.1.3 Mobile-First

8.9.2 Modalidades de Interfaz

8.9.3 Patrones de Diseño UX

8.9.4 Tendencias Web

8.9.4.1 UX Moderno

8.9.4.2 UI Moderno

8.9.4.3 Heurísticas de Nielsen

8.10 Evaluación de Diseño

Práctica Integral

8.1 El Arte del Diseño de Software

El diseño de software es el proceso de transformar el "qué" (requisitos) en el "cómo" (una estructura técnica implementable).

Objetivos Principales:

- Mantenibilidad a largo plazo.
- Eficiencia en el uso de recursos.
- Escalabilidad ante el cambio.
- Comprensibilidad para el equipo.



Concepto: Abstracción

La abstracción permite ocultar los detalles de implementación complejos para enfocarse en la funcionalidad esencial. Es la herramienta fundamental para manejar la complejidad.

Ejemplo 1: Control de
Modelamos una "Venta" sin preocuparnos inicialmente por la lógica de hilos o protocolos de red en el nivel conceptual.

Ejemplo 2: Interfaz de
Un "Botón" abstrae la lógica de dibujo en pantalla y el manejo de interrupciones del hardware.

Concepto: Modularidad

Dividir el software en componentes independientes (módulos) con interfaces claras.

Ventaja 1: Paralelismo

Diferentes equipos pueden trabajar en el módulo de "Pagos" y "Catálogo" simultáneamente si las interfaces están bien definidas.

Ventaja 2: Aislamiento

Un error en el módulo de "Reportes" no debería detener la operación del módulo de "Facturación".

| Concepto: Ocultamiento (Information Hiding)

David Parnas definió este principio como la base de la modularidad. Un módulo solo debe exponer lo necesario a través de su interfaz.

8.1.2 Interacción Diseño vs Requisitos

El diseño retroalimenta a los requisitos. A menudo, las limitaciones técnicas descubiertas en el diseño obligan a renegociar el alcance del producto.

- **Requisitos Funcionales:** Definen la estructura lógica.
- **Requisitos No Funcionales:** Dictan la arquitectura física (Uptime, Latencia).



Mapeo de Requisitos al Diseño

Técnica de Escenarios

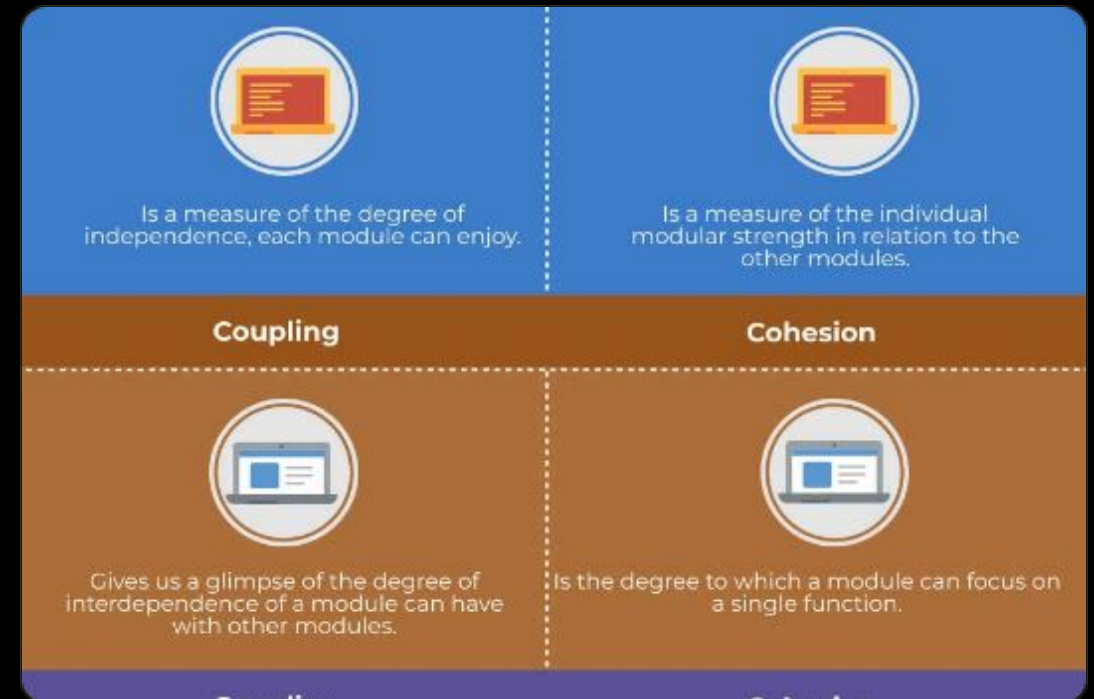
Se crean casos de prueba basados en requisitos para validar si el diseño arquitectónico propuesto puede cumplirlos eficientemente.

Requisito: Procesar 10,000 transacciones/sec.

Diseño: Se opta por una cola de mensajes (RabbitMQ) y procesamiento asíncrono.

8.1.3 Principio: Bajo Acoplamiento

El acoplamiento mide la dependencia entre módulos. El objetivo es que sea lo más bajo posible para permitir cambios locales sin efectos globales.



Tipos de Acoplamiento (De Mejor a Peor)

1. De Datos

Módulos comparten solo datos necesarios mediante parámetros.

Ej: `calculateTotal(price, tax)`.

2. De Sello (Stamp)

Módulos comparten estructuras de datos complejas.

Ej: Pasar un objeto `User` completo cuando solo se necesita el `email`.

Tipos de Acoplamiento Críticos

Acoplamiento de Control

Un módulo le dice a otro qué hacer pasando "flags" de control.

Ej: `processOrder(order, isInternational)`. Crea dependencias de lógica interna.

Acoplamiento de Contenido (Peor)

Un módulo accede y modifica los datos internos de otro módulo directamente. Rompe totalmente la encapsulación.

Principio: Alta Cohesión

La cohesión mide qué tan enfocadas están las responsabilidades dentro de un mismo módulo. Un módulo cohesivo hace "una sola cosa y la hace bien".

Cohesión Funcional (Ideal)

Todas las partes del módulo contribuyen a una única tarea bien definida.

Cohesión Coincidental (Evitar)

El módulo agrupa tareas sin relación alguna, solo por conveniencia de archivo.

Tipos de Cohesión Intermedia

Tipo	Descripción
Secuencial	La salida de una parte es la entrada de la siguiente.
Comunicacional	Partes que operan sobre los mismos datos.
Temporal	Tareas que deben ocurrir al mismo tiempo (ej: Inicio de sistema).

Principios SOLID

Acrónimo introducido por Robert C. Martin para facilitar el diseño de software mantenible y escalable.

S - Single Responsibility

O - Open/Closed

L - Liskov Substitution

I - Interface Segregation

D - Dependency Inversion



S: Responsabilidad Única (SRP)

"Una clase debería tener una única razón para cambiar".

Violación: Una clase `Invoice` que calcula el total, lo guarda en la base de datos y lo imprime en PDF. Si cambia el formato del PDF, la clase cambia. Si cambia la DB, la clase cambia.

Solución: Separar en `Invoice` (datos), `InvoiceRepository` (persistencia) e `InvoicePrinter` (presentación).

O: Abierto/Cerrado (OCP)

"Entidades de software abiertas para extensión, pero cerradas para modificación".

Técnica: Polimorfismo

En lugar de usar un `switch` gigante para calcular descuentos por tipo de cliente, creamos una interfaz `DiscountStrategy` y cada tipo de cliente implementa su propia lógica.

Resultado: Añadir un nuevo tipo de descuento no requiere tocar el código existente.

| L: Sustitución de Liskov (LSP)

"Si S es un subtipo de T, los objetos de tipo T pueden reemplazarse por objetos de tipo S sin alterar el programa".

I: Segregación de Interfaces (ISP)

"Los clientes no deben ser obligados a depender de interfaces que no utilizan".

Interfaz Gorda

Una interfaz `Worker` con `work()` y `eat()`. Un robot implementa `Worker` pero no puede comer.

Interfaz Segregada

Interfaces `Workable` y `Feedable`. El robot solo implementa `Workable`.

| D: Inversión de Dependencias (DIP)

"Depender de abstracciones, no de concreciones".

Esto permite cambiar el almacenamiento (SQL a Cloud Storage) sin tocar la lógica de negocio.

Impacto de SOLID en el Ciclo de Vida

Principio	Fase Beneficiada
SRP	Pruebas Unitarias y Debugging
OCP	Evolución y Escalabilidad
LSP	Robustez del Sistema
ISP	Desacoplamiento de Módulos
DIP	Mantenibilidad e Intercambiabilidad

8.2 Estrategia: Diseño Top-Down

Se parte del nivel de abstracción más alto (la aplicación completa) y se descompone en subniveles cada vez más detallados.

Proceso de Refinamiento

1. Definir funciones principales.
2. Dividir en subsistemas.
3. Especificar módulos de bajo nivel.

Riesgo: Se pueden ignorar limitaciones técnicas de bajo nivel hasta muy tarde en el proyecto.

8.2 Estrategia: Diseño Bottom-Up

Se diseñan primero los componentes base (librerías, utilidades) y luego se ensamblan para formar el sistema superior.

Ventaja

Componentes base muy probados y genéricos (reutilizables).

Desventaja

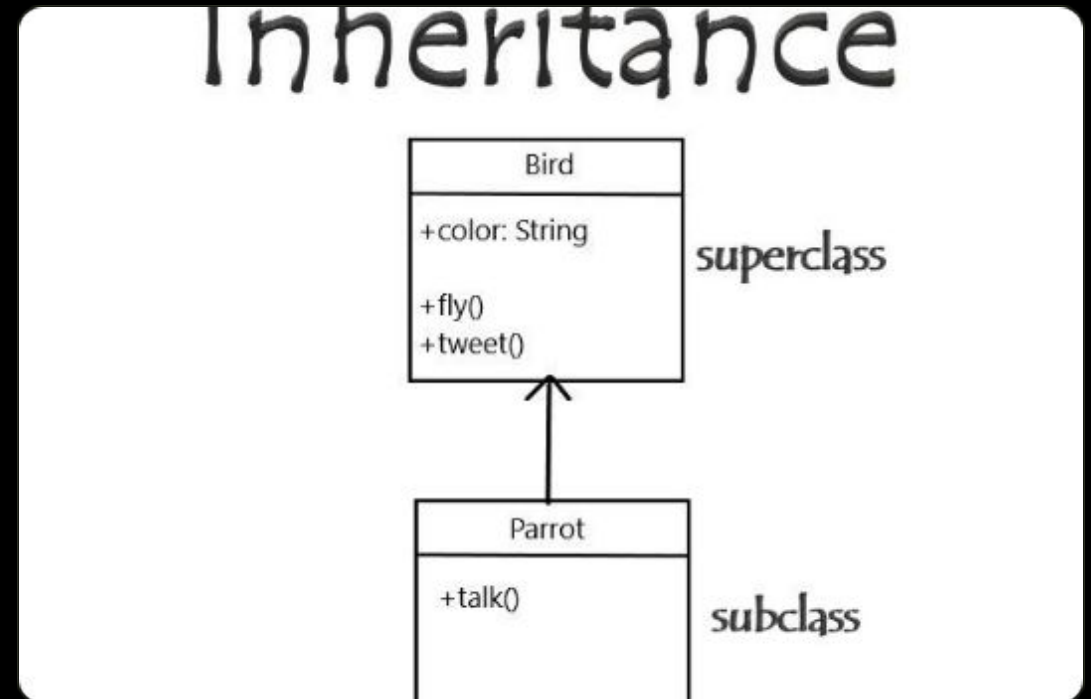
El sistema final puede no ajustarse perfectamente a las necesidades del usuario al ser una "suma de partes".

8.3 Diseño Orientado a Objetos

Visualiza el sistema como una red de objetos que colaboran enviándose mensajes.

Pilares:

- **Encapsulamiento:** Datos y métodos juntos.
- **Herencia:** Reutilización de jerarquías.
- **Polimorfismo:** Interfaces uniformes para tipos distintos.



| Colaboración entre Objetos

En el DOO, el foco no está en los datos pasivos, sino en la interacción. Los objetos tienen **Estado** (atributos) y **Comportamiento** (métodos).

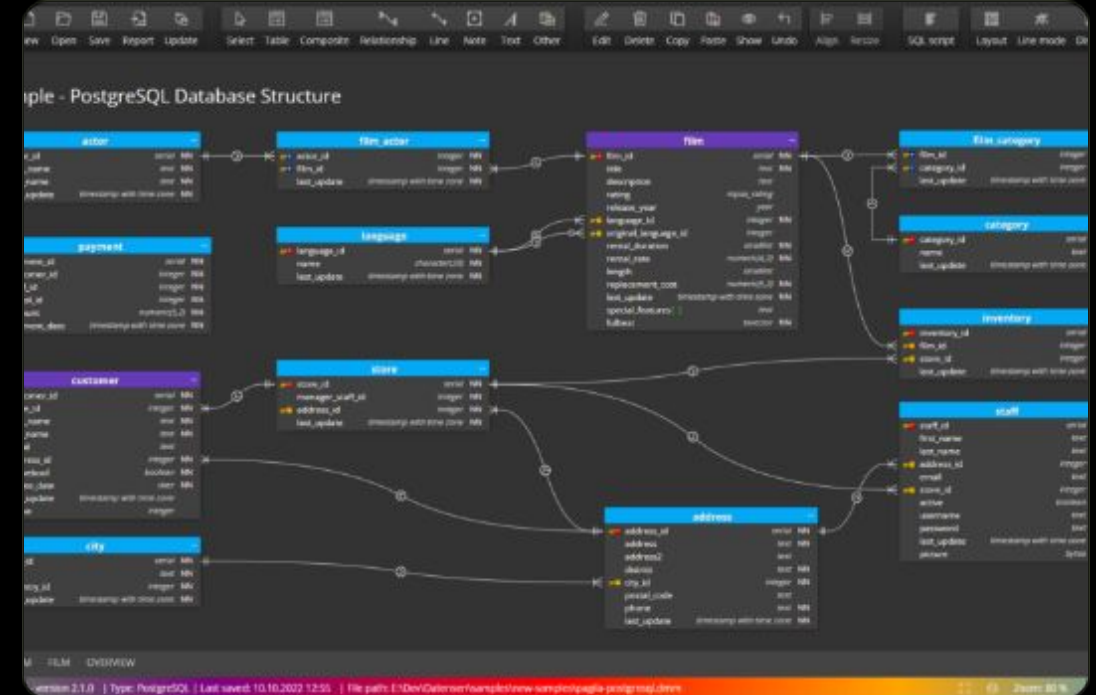
8.4 Diseño Orientado a Datos

El foco principal es la estructura de los datos y su persistencia.

Común en sistemas de gestión de información empresarial.

Etapas:

- 1 Modelado Entidad-Relación (E-R).
- 2 Normalización (Eliminar redundancia).
- 3 Esquema Físico (Indexación, Particionamiento).



Importancia de la Normalización

Integridad

Evita anomalías de actualización. Si un cliente cambia de dirección, se cambia en un solo lugar.

Desnormalización

En sistemas modernos de Big Data, a veces se prefiere repetir datos para ganar velocidad de lectura.

8.5 Trazabilidad de Requisitos

Asegura que cada requisito funcional tenga una contraparte en el diseño y viceversa.

Matriz de Trazabilidad

Un documento que mapea: [ID Requisito] -> [ID Componente de Arquitectura] -> [ID Clase de Diseño].

Utilidad: Si un requisito cambia, sabemos exactamente qué clases y módulos debemos revisar.

8.6 Patrones de Diseño (GoF)

Soluciones probadas a problemas comunes de diseño de software. Introducidos por la "Banda de los Cuatro" (Gamma, Helm, Johnson, Vlissides).

Categorías:

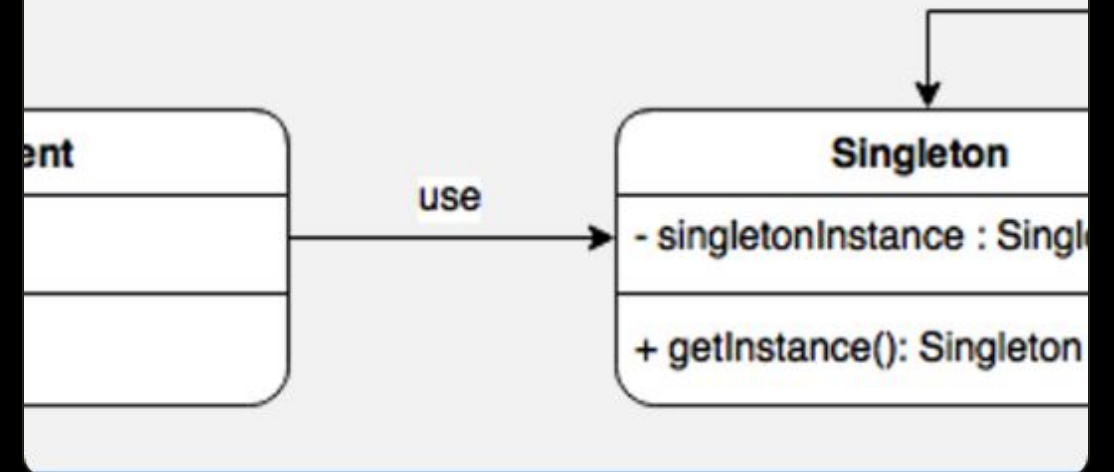
- Creacionales.
- Estructurales.
- De Comportamiento.



Patrón Singleton

Garantiza que una clase tenga una única instancia en toda la aplicación.

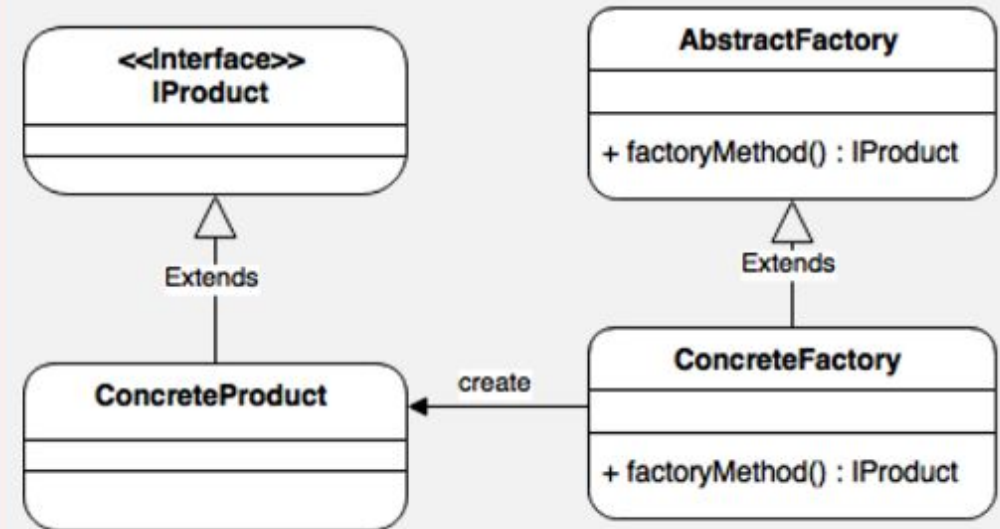
Singleton pattern – Class diagram



Patrón Factory Method

Define una interfaz para crear objetos, pero permite a las subclases decidir qué clase instanciar.

Factory Method – Class diagram



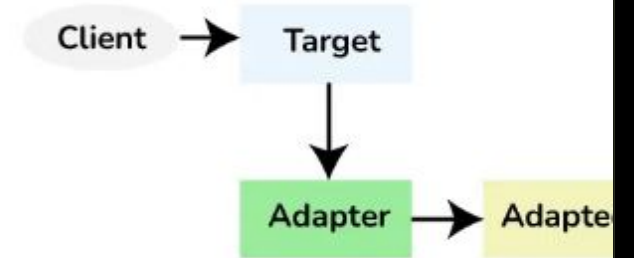
Patrón Builder

Separa la construcción de un objeto complejo de su representación, permitiendo que el mismo proceso de construcción cree representaciones diferentes.

8.6.2 Patrones Estructurales: Adapter

Convierte la interfaz de una clase en otra interfaz que el cliente espera.

Adapter
Design
Pattern



Patrón Facade (Fachada)

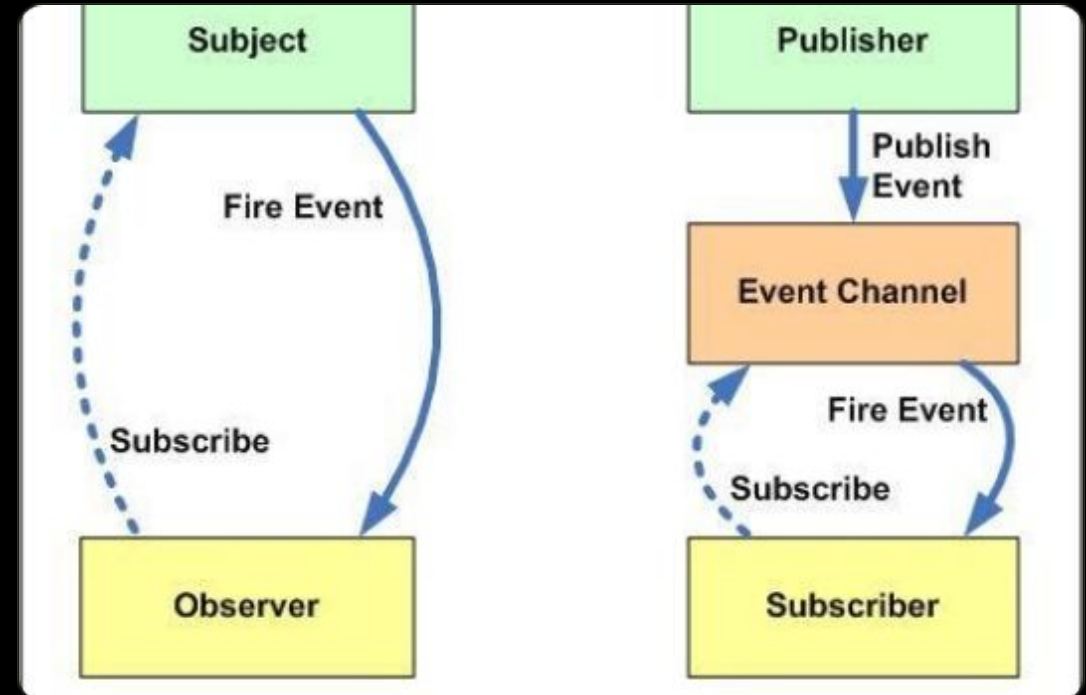
Proporciona una interfaz unificada y simple para un conjunto de interfaces en un subsistema complejo.

Caso: Sistema de Home Theater

La fachada `HomeTheaterFacade` tiene un método `watchMovie()` que internamente enciende las luces, baja el telón, arranca el proyector y pone el sonido envolvente.

8.6.3 Patrones de Comportamiento: Observer

Define una dependencia de uno a muchos para que cuando un objeto cambie de estado, todos sus dependientes sean notificados automáticamente.



| Patrón Strategy

Define una familia de algoritmos, encapsula cada uno y los hace intercambiables en tiempo de ejecución.

8.7 Sistemas Interactivos

Se centran en la comunicación entre el usuario y la máquina, o entre diferentes capas del sistema en tiempo real.

Herramientas de Modelado Dinámico:

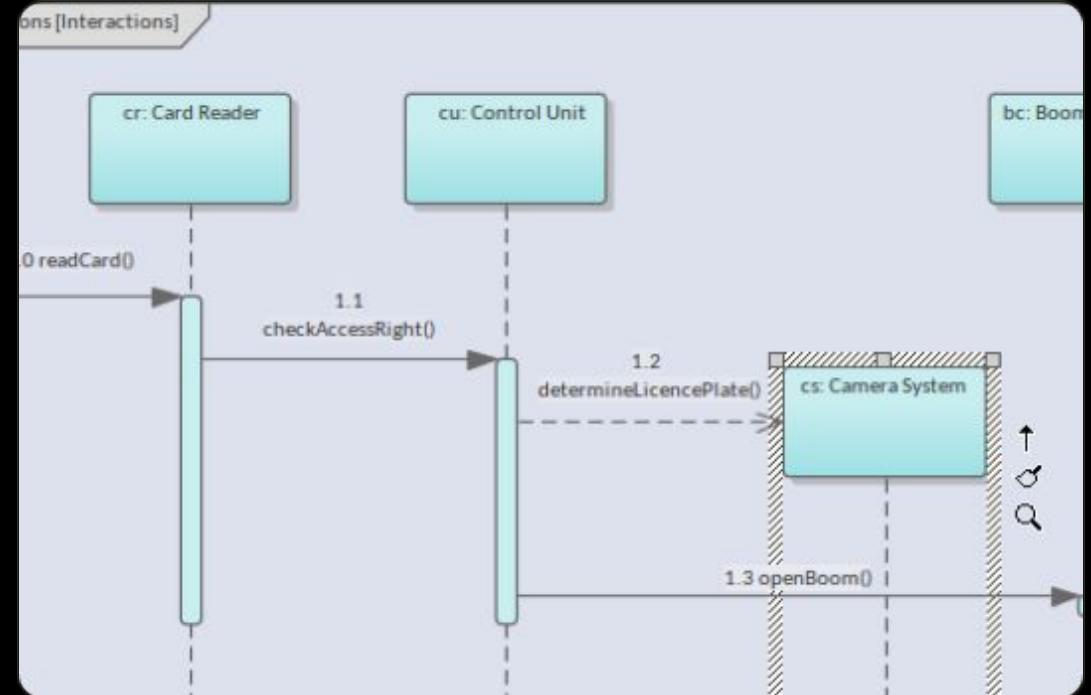
- Diagramas de Secuencia (Temporalidad).
- Diagramas de Máquina de Estado (Comportamiento reactivo).

8.7.1 Diagramas de Secuencia

Muestran la interacción de objetos organizados en una secuencia temporal. Son vitales para entender flujos de casos de uso complejos.

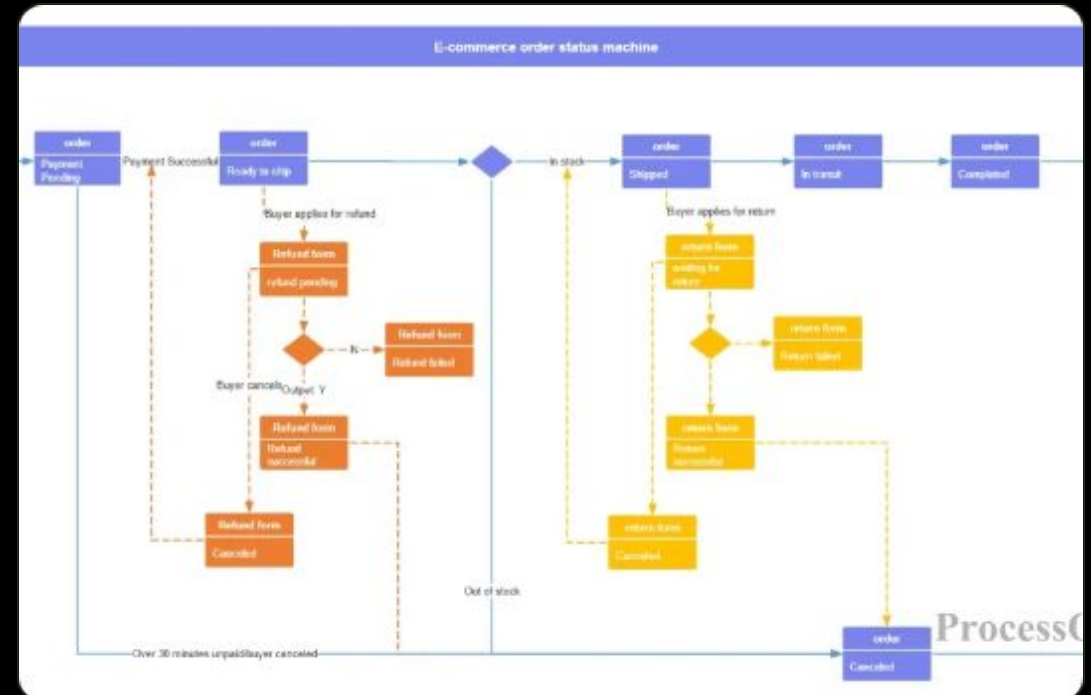
Elementos:

- Línea de vida (Lifelines).
- Mensajes (Síncronos/Asíncronos).
- Activaciones.



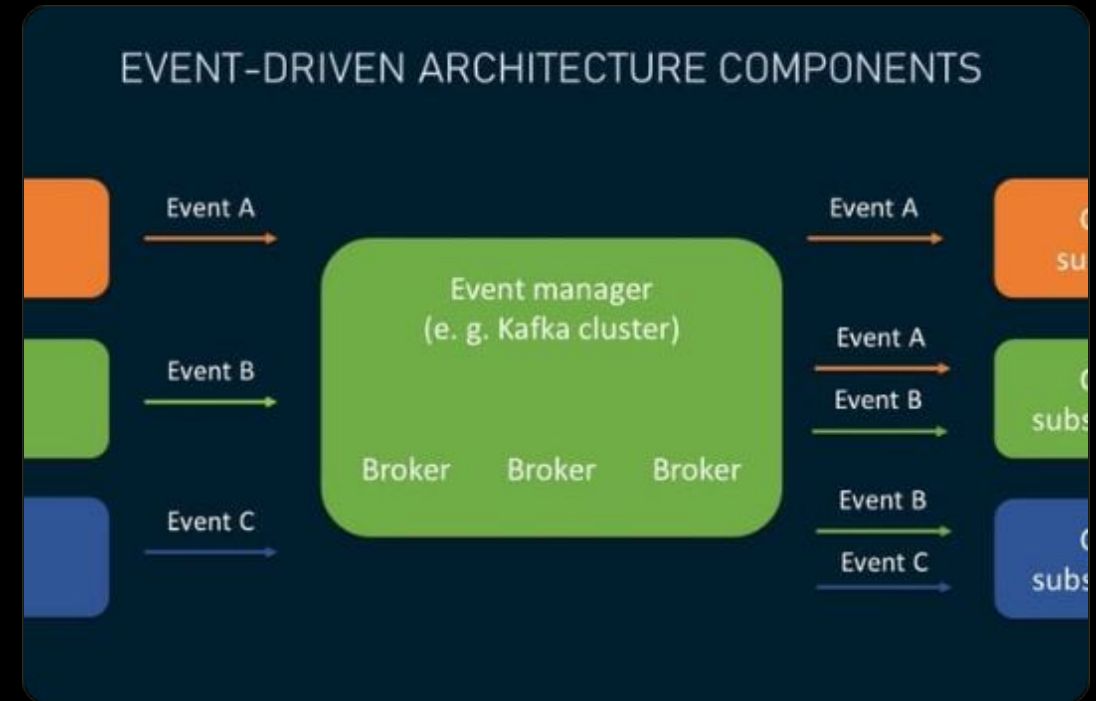
8.7.2 Máquinas de Estado

Modelan el ciclo de vida de un objeto reactivo. Un estado cambia a otro impulsado por un **Evento**.



8.8 Sistemas Impulsados por Eventos

Arquitectura desacoplada donde los componentes reaccionan a la publicación de sucesos (Eventos) en lugar de llamarse directamente.



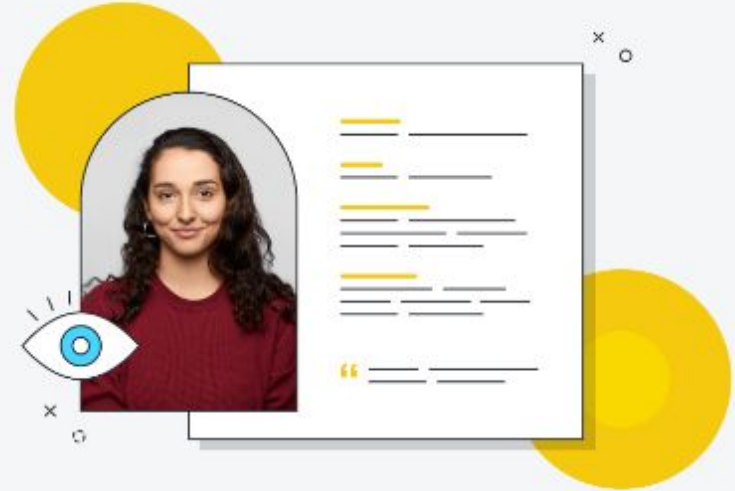
8.9 Experiencia de Usuario (UX)

UX es la percepción y respuesta de una persona al usar un producto. No es solo "colores"; es flujo, utilidad y accesibilidad.

Diferencia Clave:

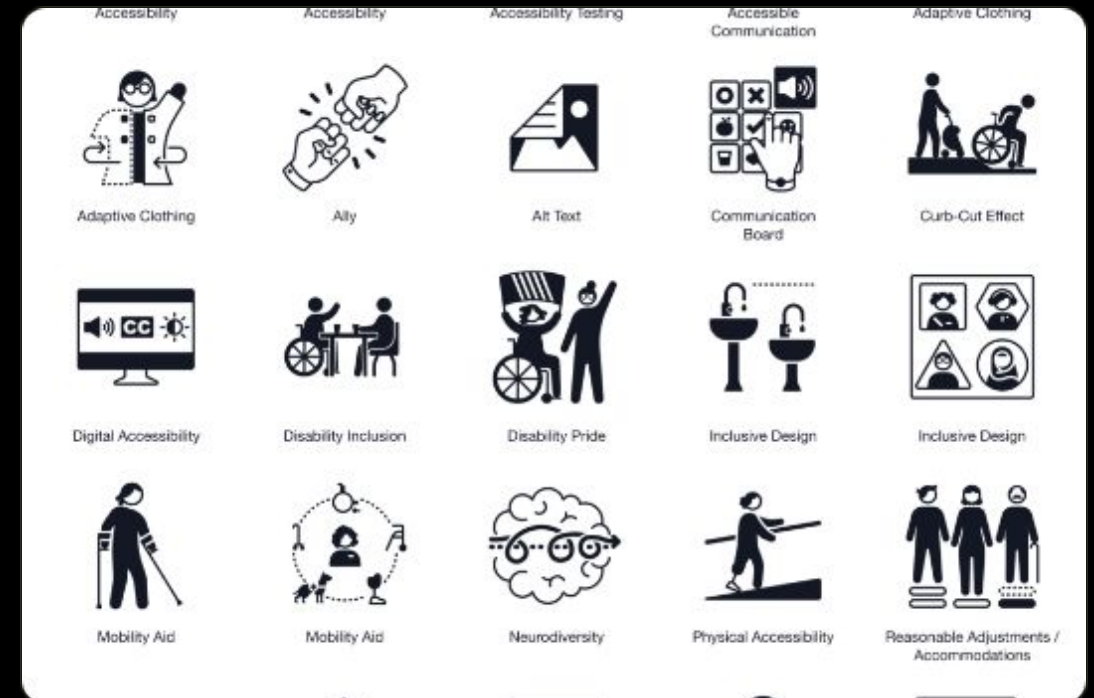
UX: Cómo funciona.

UI: Cómo se ve.



8.9.1.1 Diseño Inclusivo

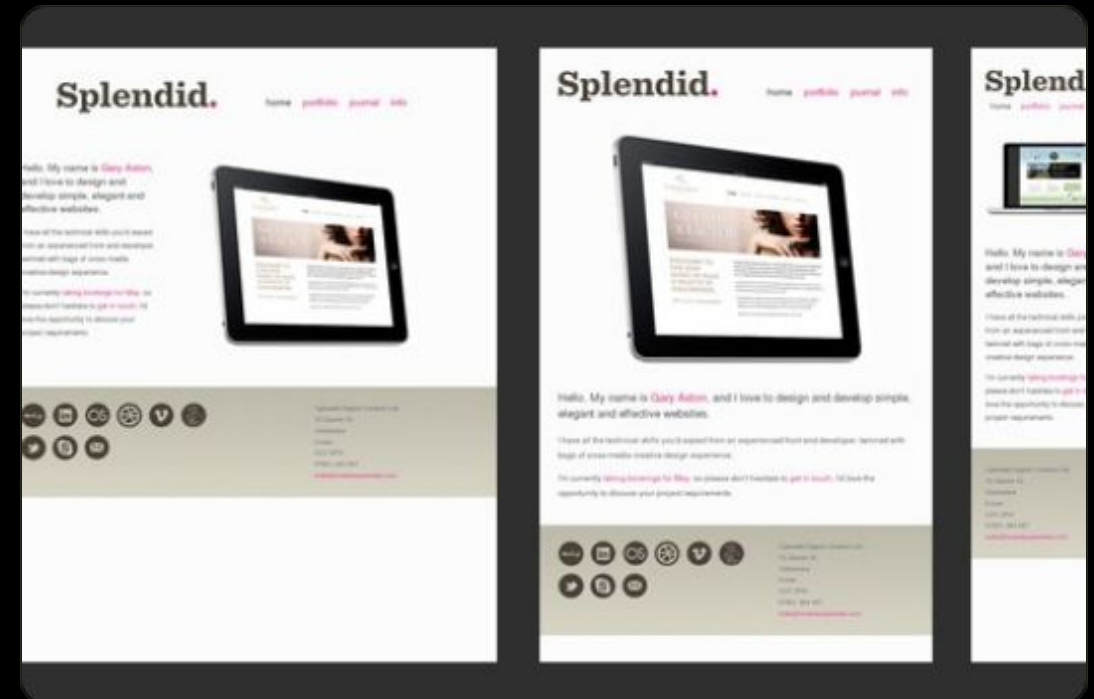
Diseñar para la diversidad humana: edad, habilidades, cultura e idioma.



8.9.1.3 Diseño Mobile-First

Estrategia de diseñar primero para la pantalla pequeña y escalar hacia arriba. Prioriza el contenido crítico.

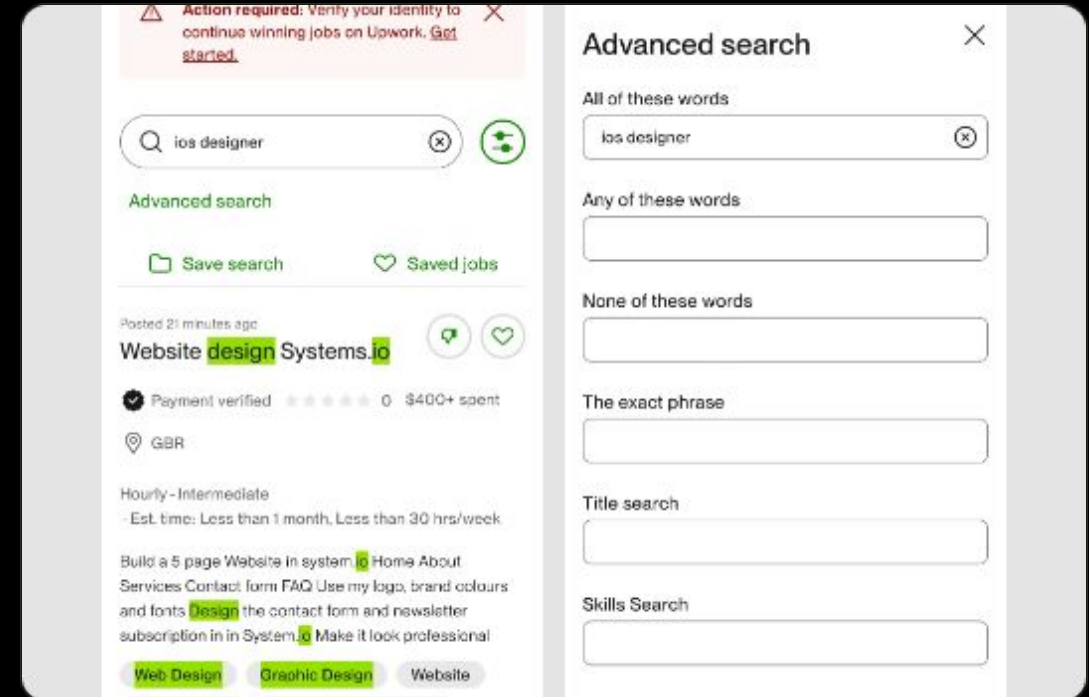
Ventaja: Limita el desorden visual (clutter) desde el inicio.



8.9.3 Patrones de Diseño UX

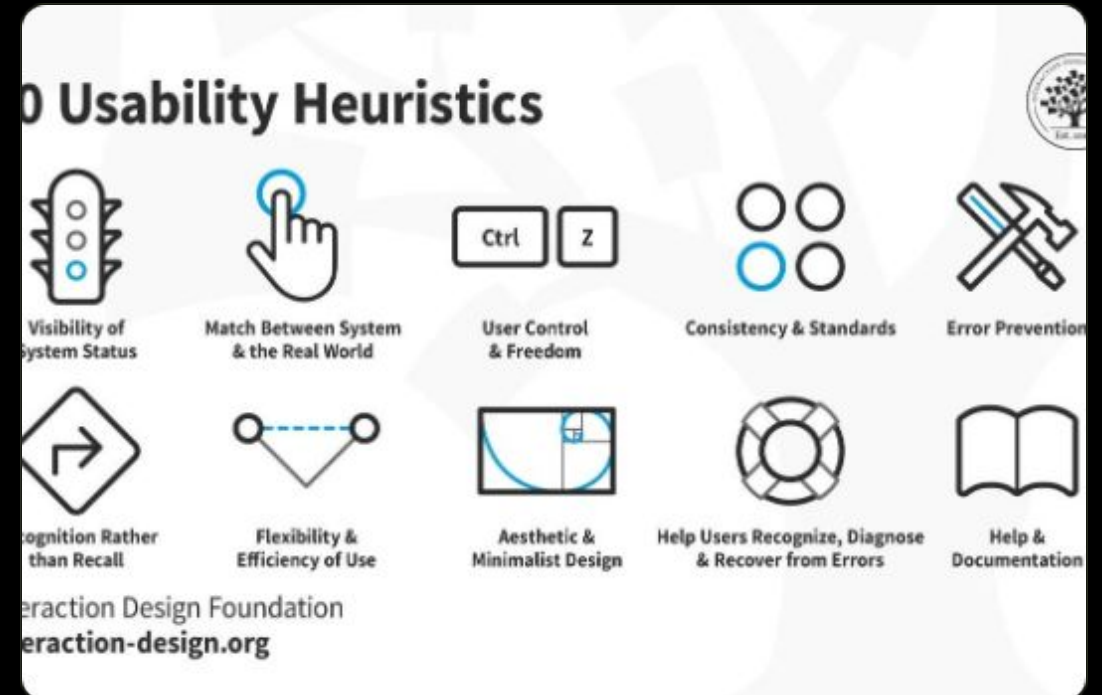
Estructuras de interfaz familiares para el usuario que facilitan el aprendizaje.

- **Onboarding:** Guía al nuevo usuario.
- **Search & Filter:** Para grandes catálogos.
- **Wizard:** Pasos guiados para procesos complejos.



Heurísticas de Jakob Nielsen (1-5)

- 1 Visibilidad del estado del sistema.
- 2 Relación entre el sistema y el mundo real.
- 3 Control y libertad del usuario.
- 4 Consistencia y estándares.
- 5 Prevención de errores.



Heurísticas de Jakob Nielsen (6-10)

- 1 Reconocer en lugar de recordar.
- 2 Flexibilidad y eficiencia de uso.
- 3 Estética y diseño minimalista.
- 4 Ayudar a los usuarios a reconocer, diagnosticar y recuperarse de errores.
- 5 Ayuda y documentación.

.

8.10 Evaluación de Diseño

Procesos para validar si el diseño cumple con los objetivos de usabilidad y funcionalidad antes del desarrollo final.

- Pruebas de Usuario (User Testing).
- Inspecciones Heurísticas.
- Recorridos Cognitivos.

